

UNIT-2

DevOps

- ① DevOps On Achieving Business Agility
- ② Continuous testing in DevOps
- ③ Monolithic Architecture, Micro Services Architecture, Architecture Rules of thumb in DevOps
- ④ Separation of Concerns in Software Architecture
- ⑤ Handling Database Migrations, Resilience in DevOps
- ⑥ Impact of DevOps on Software Quality and reliability
- ⑦ Relationship between DevOps and Software Architecture.
- ⑧ How well design Architecture Supports DevOps principles.

DevOps on Achieving Business Agility

(1)

ability to move Quickly and easily.

Business Agility :-

Business agility means a Company can easily adjust to market changes by being flexible and quick to adapt.

How DevOps helps to achieve business agility :-

DevOps is about making team work between developers and operations smoother. Automating tasks and deploying Software frequently. This helps Companies react faster to changes in market and what Customers want.

Example :-

Imagine a Software Company that used to have long waits between when developers finished their work and when it was deployed by the operations team. With DevOps they implemented automated Systems that allows developers to send their code straight to deployment. This means new features and updates can reach Customers much faster and able to adapt quickly to Customer feedback.

Examples of Companies that have adopted DevOps to achieve faster Software delivery and increased Customer responsiveness :-

Amazon:-

Amazon's DevOps practices enable them to rapidly deploy updates and new features to their e-commerce platform ^{amazon.in}. For example, during peak shopping seasons like prime Day, they smoothly handle increased traffic and transactions, ensuring a smooth customer experience.

Netflix:-

Netflix relies on DevOps to continuously enhance their streaming service.

Google:-

Google focuses on working together, being clear about what's happening, and always trying to get better to make sure their products make customers happy. They use tools that automatically check the code for any issues and quickly deploy updates.

microsoft:-

with tools like Azure DevOps, they accelerate product development cycles, allowing them to release updates to products like microsoft office and windows more frequently by using focusing on user feedback and security.

Adobe:-

Adobe utilizes DevOps to enhance its Creative Software Suite, including photoshop and illustrator.

Continuous Testing In DevOps

(2)

In DevOps, Continuous testing means testing your software non-stop, from the moment you start writing it until it's ready to place in a server. It's about automating tests so they run automatically whenever there's a change, helping to catch bugs early and ensure the software works well.

Definition and example for each type of Continuous testing:

① Unit Testing:

testing each piece of code separately to make sure it functions correctly.

② Integration testing:

Integration testing makes sure that all the different pieces of the software fit together properly and work well together.

③ Functional testing:

Functional testing ensures that the software's features and functions behave as expected.

④ Regression Testing:

Regression testing confirms that new changes haven't broken existing features.

⑤ Performance Testing:

Performance testing ensures that how well the software performs under different conditions, such as high user loads.

⑥ Security Testing:

Security testing checks the software for any weaknesses and make sure that sensitive data is protected.

Challenges:-

① Automation Complexity: Automating tests across various environments and platforms can be complex and time-consuming.

② Tool integration:

Integrating testing tools with other DevOps tools can be challenging.

③ Test Data management:

Ensuring the availability and manageability of test data for various scenarios and environments can be difficult.

⑦ Benefits:

① Early Bug Detection

② Faster Feedback Loop

③ Improved Release Confidence

④ Cost and Time Saving

⑤ Collaboration between development, testing and operations teams.

Tools Commonly Used for Continuous testing in DevOps includes

- ① JUnit & TestNG - for Unit testing
- ② Selenium & Cypress - for functional testing
- ③ Jenkins & Travis CI - for Continuous testing
automation & integration
- ④ JMeter & Gatling - for performance testing
- ⑤ OWASP ZAP & Nessus - for Security testing

Monolithic Architecture in DevOps

Monolithic architecture in DevOps refers to a traditional Software design where all components of an application like front end, Backend and Database all these are tightly integrated into a single code base, which means everything is kept together within a single folder.

Single CodeBase (Single folder or directory.)

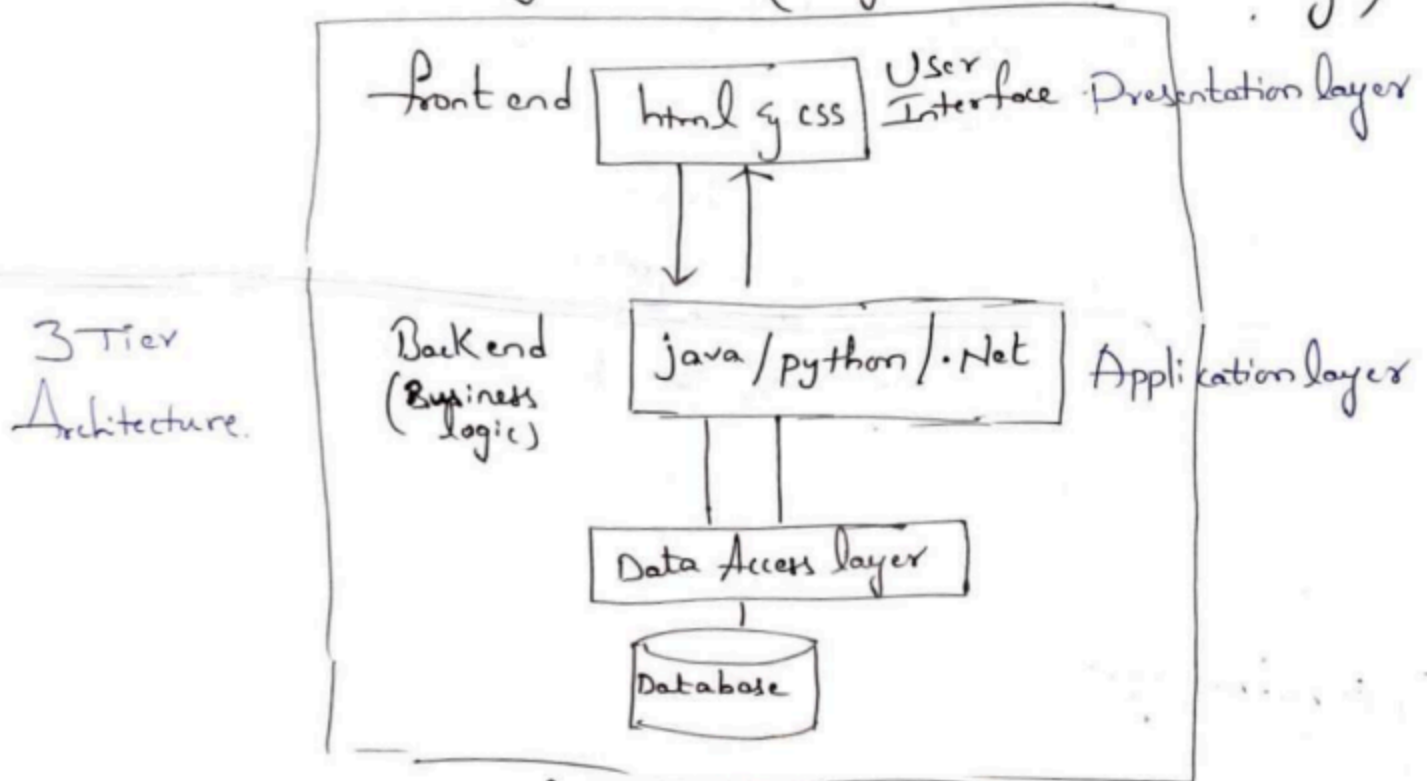


Fig: Monolithic Architecture

⇒ In monolithic Architecture, User interface, Business logic and Data Access layer works as a single unit. So, it's easier to manage all operations because all components of the software are located in one place.

⇒ In a monolithic Architecture, changes made in one layer can affect other layers because they all are tightly integrated.

For example, I created a College website. In that College website, I want to add a new feature like adding new courses. So, whenever I change the presentation layer, I need to change the business logic layer and data access layer too.

Advantages:-

- ① Simple to Develop, test and deploy Since everything is in one place
- ② Communication between components is faster since " " " "
- ③ Developers have a clear understanding of entire application as all components are tightly integrated
- ④ Deploying monolithic applications is straight-forward compared to microservices since there's only one code base to manage.

Disadvantages:

- ① it's challenging to update existing ones as the entire application needs to be modified.
- ② A fault in one part of the application can bring down the entire system since everything is tightly coupled.
- ③ as everything is tightly coupled it is difficult for team members to work independently, leading to coordination challenges

Micro Services Architecture in DevOps

If you Consider monolithic Architecture, in monolithic Architecture entire Application is placed in one Single folder. So changes made to one Component will effect other Components, but where as in micro Services Architecture your Application is placed in bunch of folders, each folder is responsible for a Specific function, So changes made in one folder will not effect other folders,

⇒ In a DevOps environment, microServices architecture shines because it allows for better Scalability, flexibility and faster deployment,

you can modify Specific part of our application independently without effecting other parts.

Teams Can choose the best tools and technologies for each microService, making it easier to adapt changing requirements

Since each microService is independent, you can deploy updates without affecting whole system

Resilience → If one microService fails, it doesn't bring down the entire application. the rest can still function normally.

These are Benefits of micro Service Architecture.

Challenges:-

Complexity:- managing multiple microServices can be more complex than monolithic Architecture.

Communication: Services need to Communicate effectively, which requires good design and Coordination

monitoring and Debugging:- with more microServices, monitoring and debugging can be challenging.

Example:-

In a college website, each function like Student attendance, Faculty attendance, Student results, College Courses can be considered as Separate Services in a micro Service architecture. changes made to one Service, Such as updating the Student attendance feature, won't affect the other Services like Faculty attendance or Student results.

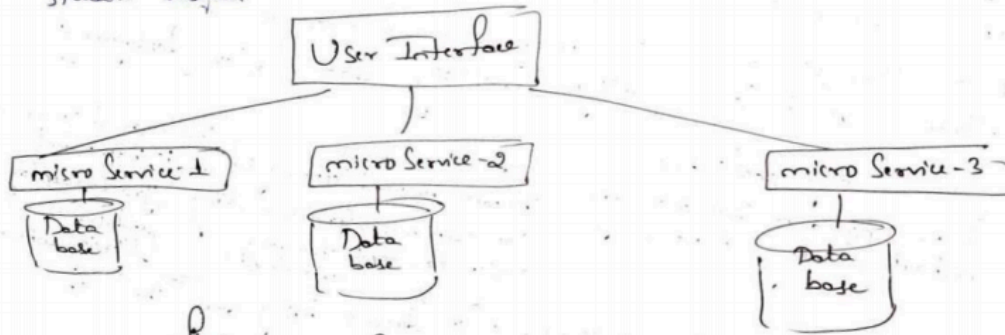


Fig. micro Service Architecture

Architectural Rules of thumb in DevOps

⇒ "Rules of thumb" are general principles or guidelines widely used in a specific area. They are not strict rules, more like practical suggestions to achieve positive results in system design and implementation.

⇒ In DevOps architecture, they represent general best practices that teams can follow to achieve effective and efficient system design.

① Modularity:- Design systems in a modular fashion (i.e., break software into smaller modules that work independently) enabling easier maintenance, updates.

② Scalability:- Ensure that architectures can handle increased load without sacrificing performance.

③ Automation:-

Automate processes wherever possible, including deployment, testing and monitoring, to increase efficiency and reduce human error.

④ Resilience:-

Resilience means building systems that can handle failures by including backup plans, automatic switches.

⑤ Infrastructure as Code (IaC):-

manage infrastructure like settings to track changes, recreate setups, and setup automatically.

⑥ Continuous integration / Continuous Deployment (CI/CD) :-

automate CI/CD process by creating pipelines

⑦ monitoring and Logging:-

Creating monitoring and logging Systems to provide check System performance, and detect issues early.

⑧ Security:-

integrate Security practices into every stage of the development and deployment process, including encryption & access control.

⑨ Containerization:-

Utilize Containerization technologies like Docker tool to package and manage applications efficiently.

⑩ Feedback loop :-

Establish a feedback loop between development, Operations and Customers to continuously improve processes and Systems based on real world usage and feedback.

Separation of Concerns in Software Architecture

Separation of Concerns (Soc) means dividing entire program into various sections based on their functionalities, so that each section runs independently without affecting other services. changes made in one section will not affect other sections.

⇒ Separation of Concerns (Soc) will improve maintainability:- Easier to update and fix parts of the code without affecting the whole system.

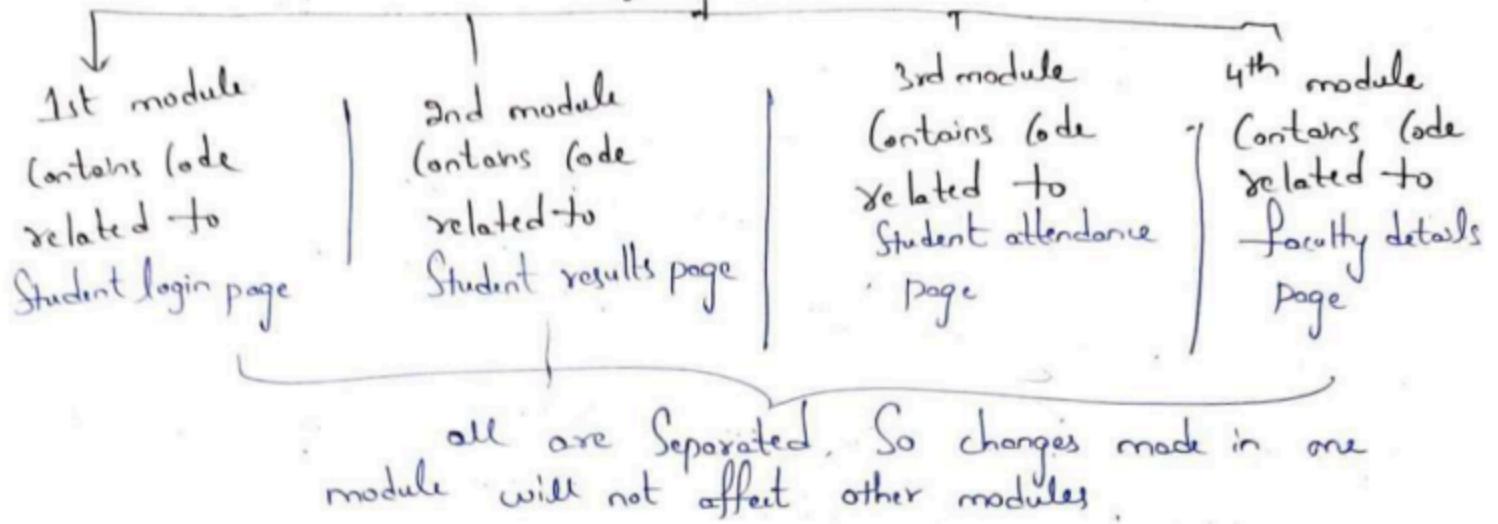
Scalability:- Simple to expand the system to handle more load and add simple to add new features.

Readability:- Code is easier to understand, making it simpler for developers to work with that code.

Debugging:- Simple to identify and fix bugs.

For example, I am creating a college application, my college application contains four pages, so I will divide it into four modules. The first module contains code related to the student login page, the second module contains code related to the student results page, the third module contains code related to the student attendance page, and fourth module contains code related to faculty details page. changes made in one module will not affect other modules, because they all are separated.

College Application



Real world Examples of SoC implementation:-

① Model-View-Controller (MVC) Architecture:-

MVC is a classic example of SoC because it divides an application into three components →

- model :- manages data and business logic
- view :- handles the presentation layer
- Controller :- intermediate interface between model & view. Process user inputs

Ex:- web applications

② Micro Services Architecture:-

In a microService Architecture, an application is built as a collection of loosely coupled services, where each service will perform certain task.

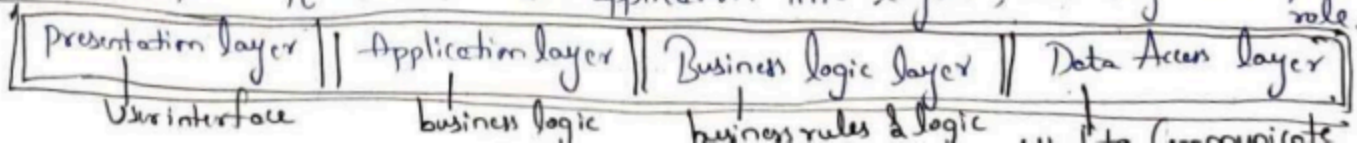
Ex:- e-commerce platforms.

③ Client - Server Architecture:-

this architecture splits an application into two main components:

client & Server → Backend that processes user requests & manages data.
 front end that interacts with users Ex:- Social media applications

④ Layered Architecture: It divides an application into layers, each layer has specific role.



Handling Database Migrations in DevOps (8)

⇒ Database migration means transferring data from one database to another. This process can not only include just moving the data itself; but also transforming it to fit the new database structure (schema), updating database structure, and ensuring data integrity and consistency throughout the migration.

⇒ In DevOps, handling database migrations means managing the process of moving data from one database to another, while ensuring it's done smoothly and without disturbances. This is important because changes to the database structure or data need to be matched with application updates to avoid errors.

Techniques Used for Database Migration in DevOps:

① Version Control :- (Git)

One of the essential techniques for database migration in DevOps is to use Version Control for both the application code and the database schema (structure). Tools like Liquibase and Flyway can be integrated into CI/CD pipelines to automatically apply database schema changes when corresponding changes are made to the code.

② Schema migration tools:-

These tools automate making and running SQL Scripts that change your database structure (schemas). They prevent mistakes and reduce downtime. Examples are SQL Server Data tools and MySQL workbench. (just write SQL script to specific changes instead of writing complete code)

③ Data migration tools:-

Sometimes, you need to move data between databases. Data migration tools make this easy and safe. They can handle big data, different formats, and keep everything in sync. Examples are AWS Database migration Services (DMS) and Azure Data Factory.

④ Testing Strategies:-

Lastly, you need to test the migrated database to make sure it works well. This includes various tests, like checking performance and security. Testing ensures the database meets all requirements and fixes any issues. Examples are SQL Test and JMeter.

Resilience in DevOps

(9)

Resilience in DevOps means the ability of Systems to recover quickly from problems and Continue working Smoothly. It ensures that Services remain available even when there are issues like failures or high traffic.

How Organizations Achieve Resilience Using DevOps :-

① Automation:-

Automate tasks like deployment and testing. Reduces human errors and Speeds up recovery

② Continuous integration / Continuous Deployment (CI/CD):-

Regularly integrate and deploy Small changes. makes it easier to find and fix issues quickly.

③ monitoring and Logging:-

mean recording detailed information about the operations of Software

Constantly check Systems for problems. Use logs to understand issues and respond faster.

④ Infrastructure as Code (Iac):-

mean hardware, Software, networks, Servers, database and other components

manage infrastructure with code. Easily rebuild or update Systems using Scripts.

⑤ micro Services Architecture:-

Break applications into Smaller, independent Services. If one Service fails, others Continue to work.

⑥ Regular Backups and Disaster Recovery plans:

Keep backups of data and Systems. Have plans to recover from major failures

⑦ Scalability:-

Design Systems to handle more load automatically.

Organizations Using DevOps for Building Resilient Architecture:

① Netflix:-

Uses automation CI/CD, and micro Services. "chaos Monkey" is a tool created by Netflix to test resilience by intentionally causing failures.

② amazon:-

Employs automation, Iac and Scalability. Ensures Services are available even during high traffic periods like "Amazon Great indian festival"

③ google:-

Utilizes monitoring, logging and automated recovery. Ensures Services like Search and Gmail are highly available